Техническое задание
Разработка финансовых отчетов и создание процедур для финансового сервиса управления платежами

Введение

Вы претендуете на позицию **MySQL-разработчика**, и в рамках данного технического задания вам предлагается продемонстрировать свои практические навыки работы с реляционными базами данных в условиях, близких к реальной высоконагруженной системе.

Тема: разработка финансовых отчетов и процедур для сервиса управления платежами.
Стек: *MySQL 8 (InnoDB, SQL/PSM, Events).*

Задание включает:

- создание хранимых процедур для обработки транзакций и ведения логов действий пользователей;
- проектирование и реализацию отчётов на основе больших объемов данных;
- разработку механизмов консолидации (агрегации) и партиционирования таблиц для повышения производительности;
- обеспечение целостности и согласованности данных при конкурентных операциях.

Основная цель задания — оценить вашу способность эффективно проектировать архитектуру БД, писать оптимизированные SQL-запросы и процедуры, использовать средства MySQL 8 (SQL/PSM, транзакции, события), а также продемонстрировать понимание того, как база данных должна масштабироваться и работать под высокой нагрузкой.

После выполнения задания вы будете должны **защитить своё решение**. На следующей встрече мы обсудим архитектурные решения, возможные улучшения, варианты оптимизации и типовые ситуации, которые могут возникнуть в реальной системе. Наша цель — оценить не только то, *как вы написали код*, но и *как вы мыслите как инженер*: как принимаете решения, видите риски, приоритезируете и объясняете свой подход.

По итогам обсуждения вашего решения будет принято решение о вашем **грейде (уровне разработчика)**. Этот результат напрямую влияет на **формирование итогового оффера** — как по позиции, так и по уровню компенсации.

Благодарим вас за то, что нашли время для выполнения этого задания. Желаем удачи и вдохновения — пусть работа над тестом будет для вас интересной и профессионально полезной.

Структура данных

Операции пользователей

Таблица содержит от 5 до 10 миллионов строк в день и хранит в себе данные за несколько лет. Содержит в себе список денежных операций пользователей финансового сервиса. Сервис работает с высоким потоком транзакций в реальном времени. Время в БД хранится в UTC. Входные параметры процедур считаются UTC.

```
CREATE TABLE payment.user_operations (
    id_operation BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    dt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    id_user INT UNSIGNED NOT NULL,
    id_type_oper SMALLINT UNSIGNED NOT NULL,
    direction TINYINT NOT NULL COMMENT 'направление движения (-1: списание, 1: начисление)',
    amount_oper DECIMAL(19, 2) NOT NULL COMMENT 'Сумма операции в валюте пользователя',
    PRIMARY KEY (id_operation)
) ENGINE = INNODB;
```

Логи действий пользователя

Таблица содержит от 5 до 10 миллионов строк в день и хранит в себе данные за несколько лет.

Содержит в себе список всевозможных действий пользователей финансового сервиса. Виды действий (поле в таблице *idAction*): пополнение счета (id=1), списание со счета (id=2).

```
CREATE TABLE payment.log_users (
dt TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
idUser INT UNSIGNED NOT NULL,
idAction INT UNSIGNED NOT NULL,
Params JSON DEFAULT NULL
) ENGINE = INNODB;
```

Справочник с пользователями финансового сервиса

Содержит в себе список пользователей, с их свойствами, пользователи выполняют денежные операции на сервисе *финансов.* Таблица содержит записи о 2 миллионах пользователей финансового сервиса.

```
CREATE TABLE payment.users (
    id_user INT UNSIGNED NOT NULL AUTO_INCREMENT,
    user_balance DECIMAL(19, 2) DEFAULT 0.00 COMMENT ‘Текущий баланс в валюте игрока’',
    id_currency SMALLINT UNSIGNED NOT NULL,
    id_country SMALLINT UNSIGNED NOT NULL,
    PRIMARY KEY (id_user)
) ENGINE = INNODB;
```

Справочник типов операций

Справочник с количеством записей до 500. Содержит в себе список всевозможных типов операций пользователей. PK из таблицы это поле *id_type_oper в таблице* payment.user_operations. Значение комиссии (type_opers.comission, в процентах) используется только для отчётов и не влияет на изменение users.user_balance.
Виды типов операций: депозит средств на счет (id=1), снятие средств со счета (id=2)

```
CREATE TABLE payment.type_opers (
	id_type_oper SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
	name_oper VARCHAR(255) DEFAULT NULL COMMENT 'Название операции',
	commission DECIMAL(5, 2) DEFAULT 0.00 COMMENT 'Процент комиссии за операцию',
	direction TINYINT NOT NULL COMMENT 'направление движения (-1: списание, 1: начисление)',
	PRIMARY KEY (id_type_oper)
) ENGINE = INNODB;
```

Справочник валют

Справочник с количеством записей до 500.
Содержит в себе список всех возможных валют с курсами к рублю. Используется как свойство пользователя. У пользователя может быть только 1 валюта.

```
CREATE TABLE payment.currencies (
	id_currency SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
	name_currency VARCHAR(255) DEFAULT NULL COMMENT 'ISO код валюты (EUR / USD / BTC...)',
	base_rate DECIMAL(15, 5) DEFAULT NULL COMMENT 'Курс к рублю',
	PRIMARY KEY (id_currency)
) ENGINE = INNODB;
```

Примечание для “base_rate”: чтобы получить эквивалент валюты в рублях, достаточно разделить базовую валюту на “base_rate”. Пример: валюта - EUR, base_rate - 0,010119000. 1 EUR / 0,010119000 = 98,8240 RUB.

Справочник стран

Справочник с количеством записей до 500. Содержит в себе список всех возможных стран. Используется как свойство пользователя. У пользователя может быть только 1 страна.

```
CREATE TABLE payment.countries (
	id_country SMALLINT UNSIGNED NOT NULL AUTO_INCREMENT,
	name_country VARCHAR(50) DEFAULT NULL,
	PRIMARY KEY (id_country)
) ENGINE = INNODB;
```

Общие требования и структура работы

В рамках данного технического задания предлагается выполнить **шесть взаимосвязанных заданий**, отражающих аспекты работы MySQL-разработчика в реальном проекте финансового сервиса.

Формат выполнения

- Каждое задание необходимо реализовать в виде **SQL-листинга** (код, создающий процедуры, события, таблицы, индексы и т.д.).
- Все решения собираются в **один текстовый файл** с названием “фамилия_имя_solution.txt”
- Готовый файл направляется **в телеграм чат для проверки и обсуждения на втором созвоне**.
- Решение должно быть выполнено на **MySQL 8**, с использованием **InnoDB**, **SQL/PSM** (хранимые процедуры, функции, обработчики ошибок) и **EVENTS** (планировщик событий).

Обязательная и дополнительная (бонусная) часть

Каждое задание состоит из двух частей:

1. **Обязательная часть** — основная задача, которую необходимо реализовать для успешного выполнения теста.
2. **Бонусная часть** — дополнительное улучшение и оптимизация, которое можно реализовать по желанию. Оно не является обязательным, но **будет плюсом** при оценке. Такие задачи отмечены в тексте как *«бонус»* и позволяют показать более глубокое понимание архитектуры, оптимизации и работы MySQL под нагрузкой.

Индексация и оптимизация

В этой части задания особое внимание уделяется **индексации** и **оптимизации запросов**. Исполнителю необходимо:

1. **Определить**, какие индексы действительно нужны для выполнения отчетов и процедур, описанных в 6 заданиях.
2. **Аргументировать** свое решение: почему индекс добавляется или не добавляется (с точки зрения фильтров, группировок и производительности).
3. Внести DDL-код добавления индексов прямо в файл решения.

Критерии оценки

При проверке решения будут учитываться:

- корректность и стабильность процедур;
- продуманность архитектуры и обоснованность индексов;
- структурированность и читаемость SQL-кода;
- способность кандидата объяснить и защитить свои решения при обсуждении;

- реализация и качество **бонусных частей** (если кандидат их выполняет по своему желанию).

Задания

Задание 1

Создать хранимую процедуру, выполняющую **зачисление** или **списание** средств со счета пользователя.

Требования к заданию:

1. Принимать параметры: **ID пользователя**, **сумма операции в валюте пользователя**, **ID типа операции** (из payment.type_opers).
2. Учитывать направление движения по полю direction в payment.user_operations: **-1 — списание**, **1 — зачисление**.
3. При списании проверять **достаточность баланса** и корректно обрабатывать недостаток средств.
4. Обновлять баланс пользователя на величину сумма × direction.
5. Записывать совершенную операцию в таблицу payment.user_operations.
6. Логировать действие в payment.log_users (поле Params — произвольный JSON по усмотрению исполнителя).
7. **Всегда** возвращать результат работы: **статус выполнения**, **баланс до** и **баланс после** операции.

Бонус

Предложить один или несколько вариантов **оптимизации** решения, чтобы сделать выполнение процедуры **максимально быстрым**, учитывая **структуру таблиц** и **объёмы данных**, указанные в условиях (индексы, порядок доступа к данным, блокировки, минимизация записей/джойнов и т.п.).

Задание 2

Реализовать хранимую процедуру финансового отчёта за произвольный период дат. Входные параметры: p_from DATETIME — начало периода, p_to DATETIME — конец периода.

Требования к заданию:

- Сгруппировать данные по **стране** пользователя и **типу операции**.
- Рассчитать метрики в **рублях** (*считая что в таблице payment.currencies курс не имеет историчности и всегда одинаковый для любых дат*):
    - amount_rub — сумма операций в рублях;
    - amount_comiss_rub — сумма комиссий в рублях;
    - amount_no_comiss_rub — итог без комиссии в рублях.
- Отчет должен содержать **промежуточные итоги по каждой стране** и **общий итог** по всем странам и типам операций.
  Сортировка: по названию страны, внутри — по типу операции; строки TOTAL выводить в конце соответствующих групп.

Бонус

- Добавить опциональные фильтры p_country_id, p_type_oper_id.
- Добавить и обработать часовой пояс как входной параметр и выводить даты в запрашиваемом часовом поясе.

Пример выполнения:

country_name	oper_name	amount	amount_comiss	amount_no_comiss
Cyprus	Deposit	10	2	8
Cyprus	Witdrawal	708	6	702
Cyprus	***TOTAL:***	718	8	710
Russia	Deposit	200000	2000	198000
Russia	***TOTAL:***	200000	2000	198000
TOTAL:	***TOTAL:***	200718	2008	198710

Задание 3

Так как таблица с операциями огромная, необходимо сделать event в виде хранимой процедуры, которая будет выполнять агрегацию данных по дню и класть эти данные в заранее созданную таблицу. Такая таблица пригодится для финансовых отчетов при выборках за большой период. Такие выборки будут эффективнее, если использовать в запросах часть данных, которая уже консолидирована, что убирает необходимость выбирать много сырых данных из огромной таблицы, и тем самым ускоряет выполнение запроса.

Требования к заданию:

1. **Создать таблицу консолидации** (структура — на усмотрение исполнителя), предназначенную для хранения дневных агрегатов.

2. **Реализовать хранимую процедуру**, которая выбирает данные из payment.user_operations за указанный день и агрегирует по **стране** и **типу операции по дню**, вычисляя агрегаты.

3. **Настроить EVENT** на ежедневный запуск в **03:00** (время сервера, UTC), который вызывает процедуру и консолидирует 1 день.

Бонус

1. **Идемпотентно** записывать результат в таблицу консолидации.
2. **Самовосстановление:** если EVENT не запускался 1–2 дня, при следующем запуске автоматически учесть пропущенные даты.

Задание 4

Реализовать хранимую процедуру отчёта за произвольный период дат, для ускорения выборки использовать таблицу консолидации из задания 3.

Входные параметры p_from DATETIME — начало периода, p_to DATETIME — конец периода.

Требования к заданию:

1. **Полные дни** внутри диапазона брать только из таблицы консолидации.
2. **Неполные дни** (хвосты в начале и конце периода) считать из payment.user_operations.
3. Группировка: **по стране** и **типу операции**, вычислить общую сумму, сумму комиссии, итоговую сумму (с вычетом комиссии) в рублях (*считая что в таблице payment.currencies курс не имеет историчности и всегда одинаковый для любых дат)*.
4. Отчет должен содержать **промежуточные итоги по каждой стране** и **общий итог** по всем странам и типам операций.
5. Вариации дат, подающихся на вход в процедуру могут быть любыми, от нескольких недель до нескольких минут.
6. Результатом выполнения задания будет листинг кода, возвращающий данные в формате, указанном в примере задания 2 (смотри картинку с таблицей).

Пример: для запроса с 2024-05-01 12:00:00 по 2024-05-05 17:00:00 нужно взять 2, 3, 4 числа из таблицы консолидации (т.к. это полные дни), а неполные дни (2024-05-01 12:00:00 - 2024-05-01 23:59:59 и 2024-05-05 00:00:00 - 2024-05-05 17:00:00) взять из сырых данных, т.е. из таблицы payment.user_operations.

Задание 5

Финансовый отчет по операциям конкретного пользователя за период дат.

Входные параметры p_from DATETIME — начало периода, p_to DATETIME — конец периода, ID пользователя типа int.

Требования к заданию:

Нужно сгруппировать данные по типу операции, вывести общую сумму, сумму комиссии, итоговую сумму (с вычетом комиссии) в рублях (*считая что в таблице payment.currencies курс не имеет историчности и всегда одинаковый для любых дат*.

Бонус

Сделать еще один входной параметр - ID валюты типа int. Дополнить нужной логикой запрос, чтобы суммы могли конвертироваться в передаваемую на вход валюту (для задания предполагаем что передает валюту USD). Если параметр не передается, конвертировать к рублям.

Задание 6

Реализовать ежедневную ротацию партиций таблицы логов payment.log_users с дневными партициями по колонке dt, чтобы хранение и запросы по свежим данным были быстрыми и предсказуемыми.

Требования к заданию:

- Перевести payment.log_users на партиционирование по дате/времени:
  - Тип: PARTITION BY RANGE COLUMNS(dt) (дневные границы).
  - Схема именования: pYYYYMMDD (рекомендуется для читаемости и удобства администрирования).
- Создать хранимую процедуру ротации, которая выполняет два действия:
  - Удаление старых партиций: удалить все партиции, чья дата строго меньше CURDATE() - INTERVAL 7 DAY.
    (Т.е. храним последние 7 «прошедших» дней + сегодняшние/будущие партиции по условиям ниже.)
  - Добавление партиций с запасом: обеспечить наличие партиций на сегодня и еще на 2 дня вперед (на случай пропуска события).
- Создать EVENT ежедневного запуска в 02:00 (время сервера), который вызывает эту процедуру.
- Повторный запуск процедуры не должен приводить к ошибкам.
  Подсказка: для удаления и создания партиций собирать DDL-запросы через prepared statements.

Бонус

Автовосстановление после простоя: если EVENT не запускался N дней, процедура добавляет все пропущенные партиции и аккуратно удаляет старые одним запуском.